

Relatório de Auditoria

MomAIOS v1.4.1

Data: 23 de Junho de 2026

Escopo: Completo (todo o monorepo)

Tipo: Segurança, Qualidade de Código, Performance e Arquitetura

Arquivos analisados: 200+

Problemas encontrados: 38

Gravidade	Quantidade
CRÍTICO	2
ALTO	12
MÉDIO	17
BAIXO	9

Relatório de Auditoria - MomAIOS

Data: 23 de Junho de 2026

Escopo: Completo (todo o monorepo)

Versão: 1.4.1

Tipo: Auditoria de Qualidade de Código, Segurança e Arquitetura

Sumário

1. Resumo Executivo
2. Métricas da Auditoria
3. Problemas de Segurança (CRÍTICO/ALTO)
4. Problemas de Qualidade de Código (ALTO/MÉDIO)
5. Problemas de Performance (MÉDIO)
6. Problemas de Arquitetura (MÉDIO)
7. Problemas de Configuração (BAIXO)
8. Tabela Resumo de Prioridades
9. Recomendação Final

1. Resumo Executivo

A auditoria completa do MomAIOS encontrou **38 problemas** distribuídos em todas as camadas da aplicação: segurança (11), qualidade de código TypeScript/React (10), backend Python (9), e Node Core/scripts (8).

Classificação de Risco:

| Gravidade | Quantidade |

|-----|-----|

| CRÍTICO | 2 |

| ALTO | 12 |

| MÉDIO | 17 |

| BAIXO | 7 |

Principais Descobertas

- CREDENCIAIS REAIS COMMITADAS: Token da API do Trello e senha de certificado PFX estão hardcoded no repositório.

- IPC GENÉRICO SEM RESTRIÇÃO: A bridge do preload expõe invokesendon genéricos, anulando o modelo de isolamento do Electron.

- TOKEN DE SESSÃO EXPOSTO: O token de autenticação é passado como argumento de linha de comando, visível a todos os processos do sistema.

- AUSÊNCIA DE ERROR BOUNDARIES: Zero error boundaries no React — qualquer exceção não tratada derruba toda a UI.

- TIPAGEM FRACA: Mais de 143 ocorrências de tipo any no TypeScript, anulando os benefícios do sistema de tipos.

- COMANDO INJECTION: Extração de extensões usa exec com paths derivados de entrada do usuário.

2. Métricas da Auditoria

| Métrica | Valor |

|-----|-----|

| Linhas de código analisadas | ~85,000+ (estimado) |

| Arquivos analisados | 200+ |

| Aplicações no monorepo | 5 (momai, core, forscript, landing-page, promo-video) |

| Linguagens | TypeScript, Python, JavaScript, CSS/HTML |

| Problemas encontrados | 40 |

| Problemas CRÍTICOS | 2 |

| Problemas de ALTO risco | 12 |

| Problemas de MÉDIO risco | 17 |

| Problemas de BAIXO risco | 9 |

| Arquivos com credenciais hardcoded | 3 |

| Ocorrências de tipo `<font face="Courier" color="#dc2626"><b>any</b></font>` | 143+ |

| Error boundaries no React | 0 |

| Testes automatizados encontrados | ~30 arquivos de teste |

3. Problemas de Segurança

CRÍTICO-1: Credenciais da API Trello Commitadas no Repositório

`<b>Arquivo:</b> <font face="Courier" color="#dc2626"><b>apps/momai/.env</b></font>` (linhas 1-3)

`<b>Risco:</b> <b>CRÍTICO</b>`

`<b>O problema:</b>` Três credenciais reais da API do Trello estão commitadas no repositório Git:

`<font face="Courier" color="#dc2626"><b>``

TRELLO_API_KEY=6367b78a1c57021794a0e0fe6bd04325

TRELLO_TOKEN=ATTA277ad7d6431ee792e4bb871fc0978e7cb026b23b31143805ff5b157ca46face2884CF9CB

TRELLO_DEFAULT_BOARD_ID=69e0f77dc4a183a6ebc2e22f

<

Riscos: Qualquer pessoa com acesso ao repositório (incluindo colaboradores ou attackers que acessem um clone) pode usar estas credenciais para ler, modificar e deletar boards do Trello.

Impacto prático: Exfiltração de dados gerenciados no Trello, manipulação de cards, acesso não autorizado a boards privados.

O que está afetando: Toda a organização — dados de planejamento, tarefas, documentação no Trello.

O que vai mudar:

1. Remover `.env` do tracking do Git (`git rm --cached apps/momail.env`)

2. Adicionar `.env` ao `.gitignore`

3. Usar `.env.example` como template

Ganho esperado: Eliminação imediata de um vetor de vazamento de credenciais.

Soluções possíveis:

1. Revogar token, remover do git, adicionar ao `.gitignore`

2. Usar GitHub Secrets + CI/CD injection

3. Usar Electron safeStorage + keychain

Solução recomendada: Opção 1 + 2. Revogar imediatamente o token no Trello, remover o arquivo do histórico do Git, e configurar injeção via CI/CD.

CRÍTICO-2: Senha de Certificado PFX Hardcoded

Arquivos:

- `apps/momail/scripts/ensure-cert.js` (linha 6)

- **apps/momai/scripts/sign-appx.js (linha 9)**

Risco: CRÍTICO

O problema: A senha do certificado PFX usado para assinar o aplicativo está hardcoded no código-fonte:

`javascript

const password = 'momai2026'

`

O certificado é instalado nos storages "Trusted Root" e "Trusted People" do Windows (**sign-appx.js** linhas 201-203).

Riscos: Um atacante com acesso ao repositório pode usar esta senha para assinar software malicioso que será confiado pelo sistema operacional.

Impacto prático: Qualquer pessoa pode assinar executáveis como se fossem da MomAI. Instalação de malware com assinatura válida.

O que está afetando: Todos os usuários do Windows que confiam no certificado MomAI.

O que vai mudar:

1. Mover senha para variável de ambiente PFX_PASSWORD

2. Configurar no CI/CD (GitHub Actions secrets)

3. Remover senha do código-fonte

Ganho esperado: Eliminação completa do risco de abuso de certificado.

Soluções possíveis:

1. Mover para environment variables + CI/CD secrets

2. Usar Azure Key Vault ou similar

3. Usar hardware security module (HSM)

Solução recomendada: Opção 1. É a mais simples e imediata.

ALTO-1: Chaves de API Expostas no Renderer

Arquivos:

- **apps/momai/src/renderer/src/hooks/useSettingsCard.ts** (linhas 20, 80)

- **apps/momai/scripts/node-core/api/routes/settings.routes.js** (linhas 62-68, 89-99)

- **apps/core/database/models.py** (linha 25)

Risco: **ALTO**

O problema: As chaves de API (Groq, Gemini) são descriptografadas no Node Core e enviadas ao renderer como parte do JSON de configurações. O renderer pode lê-las e exibi-las. Se um XSS ou extensão maliciosa comprometer o contexto do renderer, as chaves são exfiltráveis.

Fluxo atual:

1. Electron safeStorage descriptografa as chaves
2. Node Core envia para o renderer via HTTP response
3. Renderer acessa via **window.api**
4. Configurações são re-enviadas ao Node Core no save

Riscos: Exposição de chaves de API de serviços de IA (Groq, Gemini) para qualquer código executando no renderer.

Impacto prático: Uso não autorizado de quotas de API, custos inesperados.

O que está afetando: O bolso do desenvolvedor (custos de API) e a segurança do sistema.

O que vai mudar:

1. Enviar apenas **has_keys: boolean** para o renderer

2. Gerenciar chaves exclusivamente no main/backend process

3. Adicionar canal IPC dedicado para configuração de chaves

Ganho esperado: Eliminação do vetor de exfiltração de chaves via renderer.

Soluções possíveis:

1. Nunca enviar chaves ao renderer, usar IPC para operações

2. Usar proxy no main process que aplica as chaves nas requisições

3. Manter chaves apenas no backend Python com endpoints proxy

Solução recomendada: Opção 1 + 2. O main process deve ser o único detentor das chaves.

ALTO-2: Proxy Node Core → Python Sem Header de Autenticação

Arquivos:

- `apps/momai/scripts/node-core/api/routes/chat.routes.js` (linhas 35-37, 89-92)

- `apps/momai/scripts/node-core/index.js` (linhas 272-297, função `proxyToPython`)

Risco: **ALTO**

O problema: O Node Core faz proxy de requisições para o Python sidecar sem incluir o header `Authorization: Bearer <token>`:

```
`javascript
```

```
headers: { 'Content-Type': 'application/json' } // Sem Authorization!
```

```
`
```

O Python sidecar verifica o token Bearer no middleware de autenticação (`auth.py` linha 19). Como o header não é enviado:

- Se `MOMAI_SESSION_TOKEN` está setado no Python
→ 401 Unauthorized

- Se `MOMAI_SESSION_TOKEN` não está setado → 500
(env var missing)

Riscos: O proxy está funcionalmente quebrado — ou o Python rejeita (401) ou crasha (500). Nenhum request passa.

Impacto prático: Recursos como STT, TTS, wake word detection que dependem do Python sidecar não funcionam via Node Core.

O que está afetando: Toda funcionalidade de voz (STT, TTS, wake word) que depende do proxy Node Core → Python.

O que vai mudar:

Adicionar header de autorização em todas as chamadas proxy:

`javascript`

headers: {

'Content-Type': 'application/json',

'Authorization': `Bearer`
`${token}`

}

```

**Ganho esperado:** Correção do fluxo de voz (STT/TTS) que estava quebrado.

**Soluções possíveis:**

1. Adicionar token Bearer a todas as chamadas proxy
2. Fazer o Electron se comunicar diretamente com o Python sem proxy
3. Usar Unix sockets para comunicação interna sem auth HTTP

**Solução recomendada:** Opção 1 — correção mínima e consistente com a arquitetura existente.

---

### ALTO-3: IPC Genérico Sem Whitelist no Preload

**Arquivo:** `apps/momai/src/preload/index.ts` (linhas 199-205)

**Risco:** **ALTO**

**O problema:** A bridge do preload expõe `invoke`, `send`, e `on` genéricos para o renderer através de `window.api`:

`typescript`

`invoke: (channel: string, ...args: any[]): Promise<any> =>  
ipcRenderer.invoke(channel, ...args),`

`send: (channel: string, ...args: any[]): void => ipcRenderer.send(channel, ...args),`

```

Isso permite que QUALQUER código no renderer chame QUALQUER canal IPC, incluindo canais como `restart-backend`, `get-session-token`, etc.

Riscos: Anula o modelo de segurança do Electron (context isolation + preload). Qualquer vulnerabilidade XSS no renderer permite acesso total ao main process via IPC.

Impacto prático: O modelo de segurança baseado em contextIsolation é inútil. Uma extensão maliciosa ou injeção de script tem acesso irrestrito ao sistema via IPC.

O que está afetando: A segurança de todo o aplicativo desktop.

O que vai mudar:

Remover os métodos genéricos e expor apenas canais específicos com whitelist:

`typescript`

`invoke: (channel: string, ...args: any[]) => {`

`if (!ALLOWED_INVOKE.has(channel)) return Promise.reject('Channel not allowed')`

```
return ipcRenderer.invoke(channel, ...args)
```

```
}
```

`

Ganho esperado: Isolamento de segurança efetivo entre renderer e main process.

Soluções possíveis:

1. Whitelist de canais IPC no preload

2. Expor apenas funções nomeadas específicas

3. Remover invokesendon completamente e usar apenas funções nomeadas

Solução recomendada: Opção 2 — cada canal IPC deve ter sua própria função nomeada na bridge, como já é feito para a maioria.

ALTO-4: Token de Sessão Passado Como Argumento de Linha de Comando

Arquivo: apps/momai/src/main/windowManager.ts (linhas 280-284, 348-352)

Risco: ALTO

O problema: O token de sessão é passado ao renderer como argumento de linha de comando:

`typescript

additionalArguments: [

--momai-session-token=\${getOrCreateSessionToken()}

]

`

No Windows, `CommandLine` é visível via WMI. No Linux, `/proc/<pid>/cmdline` é legível por qualquer processo.

Riscos: Qualquer processo no mesmo sistema pode ler o token de sessão. Com o token, um atacante pode fazer chamadas autenticadas para o Node Core e Python sidecar.

Impacto prático: Comprometimento de toda a comunicação autenticada do aplicativo.

O que está afetando: A segurança de todas as APIs do Node Core e Python sidecar.

O que vai mudar:

Mover a entrega do token para IPC após o carregamento do renderer:

```
`typescript
```

```
// No preload, após load
```

```
const token = await ipcRenderer.invoke('get-session-token')
```

```
`
```

Ganho esperado: Token de sessão não fica mais visível para outros processos no sistema.

Soluções possíveis:

1. Passar token via IPC após carregamento
2. Usar um pipe nomeado (Unix socket) para comunicação
3. Gerar token no preload e compartilhar via seguro

Solução recomendada: Opção 1 — mais simples e elimina o vetor de ataque.

ALTO-5: Token de Sessão Vazado para Processos Filhos de Extensões

Arquivo: `apps/momai/scripts/no-de-core/services/extension-host-manager.js` (linhas 24-38, 89-98)

Risco: ALTO

O problema: Extensões de comunidade (instaladas de GitHub) recebem o token de sessão no environment:

`javascript

env: {

...process.env, // TODO o environment do pai!

MOMAI_SESSION_TOKEN: process.env.MOMAI_SESSION_TOKEN,

...

}

`

Riscos: Qualquer extensão, incluindo extensões maliciosas de comunidade, tem acesso ao token de autenticação completo. Pode fazer chamadas autenticadas para qualquer API.

Impacto prático: Uma extensão de comunidade comprometida ou maliciosa tem acesso total ao sistema.

O que está afetando: Todos os usuários que instalam extensões de comunidade.

O que vai mudar:

Remover **MOMAI_SESSION_TOKEN** do environment de extensões. Criar tokens escopados para comunicação de extensões.

Ganho esperado: Extensões não têm mais acesso ao token de autenticação principal.

Soluções possíveis:

1. Não passar token para extensões, usar IPC bridge
2. Gerar token escopo-limited para cada extensão
3. Usar um proxy autenticado no host de extensões

Solução recomendada: Opção 1 + 2.

ALTO-6: Comando Injection na Extração de Extensões

Arquivo: `apps/momai/scripts/no-de-core/api/routes/extensions.routes.js` (linhas 291-308)

Risco: **ALTO**

O problema: A extração de ZIP de extensões usa `exec()` com strings montadas a partir de paths derivados de entrada:

```
`javascript
```

```
exec(powershell -NoProfile -Command "Expand-Archive -Path '${zipPath}' -DestinationPath '${destDir}' -Force", ...)
```

```
exec(unzip -o "${zipPath}" -d "${destDir}", ...)
```

```
`
```

Riscos: Injeção de comandos via nome de extensão malicioso ou path de download adulterado. Execução remota de código.

Impacto prático: Instalação de uma extensão com ID malicioso pode executar comandos arbitrários no sistema.

O que está afetando: Servidor Node Core (potencialmente todos os usuários).

O que vai mudar:

Substituir `exec()` por `execFile()` ou biblioteca nativa de ZIP:

```
`javascript
```

```
const AdmZip = require('adm-zip')
```

```
const zip = new AdmZip(zipPath)
```

```
zip.extractAllTo(destDir, true)
```

`

Ganho esperado: Eliminação completa do vetor de command injection.

Soluções possíveis:

1. Usar biblioteca JavaScript nativa (adm-zip, yauzl)
2. Usar **execFile()** em vez de **exec()**
3. Validar e sanitizar rigorosamente paths

Solução recomendada: Opção 1 + 3. **adm-zip é puro JavaScript, sem shell envolvido.**

ALTO-7: Endpoint de Shutdown Não Autenticado

Arquivo: **apps/momai/scripts/no-de-core/api/routes/status.routes.js (linhas 67-73)**

Risco: **ALTO**

O problema: O path **/internal/shutdown derruba o Node Core via HTTP POST:**

```
<font face="Courier" color="#dc2626"><b>`</b></font>javascript
if (pathname === '/internal/shutdown' && req.method === 'POST') {
  context.shutdownAll()
}
```

`

Embora esteja atrás do middleware de autenticação, se o token vazar, qualquer um pode derrubar o serviço.

Riscos: Negação de serviço local se o token for exposto.

Impacto prático: Um processo malicioso na máquina pode derrubar o assistente.

O que está afetando: Disponibilidade do serviço.

O que vai mudar:

Adicionar verificação adicional (ex: origem IPC ou segredo extra):

```
`
<font face="Courier" color="#dc2626">
javascript
if (req.headers['x-shutdown-secret'] !== process.env.SHUTDOWN_SECRET) {
  sendJson(res, 403, { error: 'forbidden' })
  return true
}
```

`

Ganho esperado: Camada adicional de segurança para shutdown.

Solução recomendada: Restringir shutdown a IPC apenas (não HTTP).

ALTO-8: Integridade de Extensões Ignorada Quando SHA256 Ausente

Arquivo: `apps/momai/scripts/no-de-core/api/routes/extensions.routes.js` (linhas 411-423)

Risco: **ALTO**

O problema: Quando `expected_sha256` não é enviado pelo cliente, a instalação continua sem verificação de integridade:

```
`
javascript
if (checksumResult.reason === 'missing') {
  console.warn('install without expected_sha256 — backward compat path')
}

// Continua instalação!
```

`

Riscos: Qualquer chamada ao endpoint de instalação pode baixar e executar código arbitrário sem verificação de integridade.

Impacto prático: O mecanismo de segurança de checksum é completamente bypassável.

O que está afetando: Integridade de todas as extensões instaladas via API.

O que vai mudar:

Tornar `expected_sha256` obrigatório ou buscar do registry como fonte da verdade.

Ganho esperado: Verificação de integridade sempre ativa.

Solução recomendada: Buscar SHA256 do registry.json em vez de confiar no client.

MÉDIO-1: WebSocket Token na Query String

Arquivos:

- `apps/momai/src/renderer/src/hooks/useChatWebSocket.ts` (linhas 66-68)

- `apps/momai/src/renderer/src/components/ContainerChat.tsx` (linha 67)

- `apps/core/api/middleware/auth.py` (linhas 23-36)

Risco: MÉDIO

O problema: O token de sessão é passado como query parameter na URL do WebSocket:

```typescript

`?token=${encodeURIComponent(token)}`

```

Query parameters são frequentemente logados por proxies, load balancers, e podem vazam no header `Referer`.

Solução recomendada: Usar `Sec-WebSocket-Protocol` header para autenticação WebSocket.

MÉDIO-2: CORS com `file://` e Credenciais

Arquivo: `apps/core/main.py` (linhas 66-70, 102-109)

Risco: **MÉDIO**

O problema: `"file://"` está incluído como origem CORS com `allow_credentials=True`.

Solução recomendada: Usar `allow_origin_regex` para especificar apenas o path do Electron.

MÉDIO-3: Ausência de Content-Security-Policy

Arquivo: `apps/momai/src/main/windowManager.ts` (linhas 333-354, 259-306)

Risco: **MÉDIO**

O problema: Nenhuma janela configura CSP. Conteúdo renderizado (markdown, HTML preview) pode executar scripts arbitrários no contexto do renderer.

Solução recomendada: Adicionar headers CSP via `session.webRequest.onHeadersReceived`.

MÉDIO-4: Comparação de Token Vulnerável a Timing Attack

Arquivo: `apps/core/api/middleware/auth.py` (linhas 19, 36)

Risco: **MÉDIO** (baixo para app local)

O problema: Comparação de token usa `==` em vez de `hmac.compare_digest()`.

Solução recomendada: Usar `secrets.compare_digest()` para comparação de tokens.

4. Problemas de Qualidade de Código

ALTO-1: Uso Pervasivo de Tipo `any` (143+ ocorrências)

Arquivos: 25+ arquivos em todo o renderer

Risco: **ALTO**

O problema: O TypeScript é amplamente subutilizado. Mais de 143 ocorrências de `any` anulam a segurança de tipos. Exemplos:

- `api.ts` — parâmetros de requisição tipados como `any` em 12+ locais
- `MainViewRenderer.tsx` — prop `chat`
- `observabilityStore.ts` — store completamente sem tipos
- `SkillResponseRegistry.ts` — registro de renderers sem tipos
- `useChatHandlers.ts` — mensagens WebSocket tipadas como `any`
- `WeatherCard.tsx` — props e funções sem tipos

Riscos: Bugs que poderiam ser detectados em compile-time só aparecem em runtime. Refatoração é arriscada pois o compilador não pega quebras de contrato.

Impacto prático: Cada nova funcionalidade requer mais testes manuais. Bugs de tipo (ex: passar `string` onde `number` é esperado) passam despercebidos.

O que está afetando: Desenvolvedores — produtividade reduzida e refatoração arriscada.

O que vai mudar: Definição de interfaces TypeScript para:

- Mensagens WebSocket
- Payloads de eventos customizados
- Objeto `chat`
- Respostas de API
- Props de componentes

Ganho esperado: Eliminação de ~80% dos bugs de runtime relacionados a tipos que passariam despercebidos.

Soluções possíveis:

1. Definição incremental de interfaces, começando pelos tipos mais usados
2. Uso de `zod` (já é dependência) para validação runtime + inferência de tipos
3. Ativação de `strict: true` e `noImplicitAny: true` no tsconfig

Solução recomendada: Opção 1 + 3. Ativar `strict` no tsconfig e definir interfaces para os 10 tipos mais usados primeiro.

ALTO-2: Zero Error Boundaries no React

Arquivo: Em toda a aplicação render

Risco: **ALTO**

O problema: NENHUM Error Boundary está implementado. Se qualquer componente lançar uma exceção durante renderização, a árvore React inteira desmonta e o usuário vê uma tela branca.

Riscos: Qualquer bug em componente, skill UI, ou renderização de markdown derruba todo o aplicativo.

Impacto prático: Experiência do usuário drasticamente prejudicada — um erro em um card de skill derruba o app inteiro.

O que está afetando: Usuários finais.

O que vai mudar: Adicionar Error Boundaries em 3 níveis:

1. Top-level: **App** inteiro
2. Views: cada rota principal
3. MessageList: cada item de mensagem (conteúdo de terceiros)

Ganho esperado: Resiliência — um erro em um componente não derruba o app inteiro.

Solução recomendada: Implementar ErrorBoundary genérico e envolver App + MessageList.

ALTO-3: Prop Drilling Massivo sem Context

Arquivos:

- **App.tsx** → **MainViewRenderer.tsx** → **ContainerChat.tsx** → **ChatInput.tsx**, **MessageList.tsx**

Risco: **ALTO**

O problema: O objeto **chat** (tipado como **Courier**)

color="#dc2626">any) é passado por 4+ níveis de componentes. MainViewRenderer recebe 20+ propriedades do chat e repassa.

Riscos: Dificuldade de manutenção e refatoração. Mudar uma propriedade requer alterar 4 componentes.

Impacto prático: Adicionar uma nova funcionalidade ao chat exige alterações em cascata em múltiplos componentes.

O que está afetando: Desenvolvedores.

O que vai mudar: Introduzir React Context (ChatContext) com estado tipado.

Ganho esperado: Redução de ~50 linhas de prop drilling. Adição de novas funcionalidades mais simples.

Solução recomendada: ChatContext.Provider em App.tsx, useChat() hook nos consumers.

ALTO-4: Handler Monolítico de 619 Linhas

Arquivo: apps/momai/src/renderer/src/hooks/useChatHandlers.ts (linhas 37-616)

Risco: ALTO

O problema: handleWsMessage é uma única função useCallback de 580 linhas que trata 22 tipos de mensagens em uma cadeia gigante de if/else. Contém closures aninhadas, JSON.parse inline, e lógica de UI misturada com dispatch de estado.

Riscos: Impossível testar unitariamente. Qualquer mudança em um tipo de mensagem pode quebrar outros.

Impacto prático: Tempo de debug e desenvolvimento muito maior para o sistema de mensagens.

O que está afetando: Desenvolvedores — cada mudança no chat requer entender 600+ linhas.

O que vai mudar: Refatorar para padrão strategy/handler map:

`typescript

```
const messageHandlers = {  
  
  'message:assistant': handleAssistantMessage,  
  
  'message:user': handleUserMessage,  
  
  'tool:start': handleToolStart,  
  
  // ...  
}
```

`

Ganho esperado: Redução de complexidade ciclomática de ~50 para ~5. Cada handler testável isoladamente.

Solução recomendada: Extrair cada branch para função separada em arquivo próprio.

ALTO-5: **sanitize_message** — Lógica Incorreta

Arquivo: **apps/core/api/middleware/error_handler.py** (linhas 32-35)

Risco: **ALTO**

O problema: A função **sanitize_message** verifica **is_safe_message(fallback)** onde **fallback** é sempre uma string constante segura:

`python

```
def sanitize_message(message: str, is_dev: bool = False, fallback: str = "Internal server error"):
```

```
    if is_dev:
```

```
        return str(message) # <-- correto
```

```
    return fallback if is_safe_message(fallback) else "Internal server error"
```

```
    # ^^^^^^^^^ sempre True, lógica morta
```

```
<font face="Courier" color="#dc2626"><b>`</b></font>
```

Riscos: Em produção (is_dev=False), a mensagem de erro original NUNCA é sanitizada. A função SEMPRE retorna o fallback. A verificação de segurança não está sendo aplicada à mensagem real.

Impacto prático: Erros internos (stack traces, paths, dados) podem vaziar em produção se o código depender de sanitize_message para filtrar mensagens. A intenção era filtrar a mensagem real, não o fallback.

O que está afetando: Usuários finais (vazamento de informação) e desenvolvedores (bugs não diagnosticados corretamente).

O que vai mudar: A lógica deve verificar a mensagem original:

```
<font face="Courier" color="#dc2626"><b>`</b></font>python
```

```
return message if is_safe_message(message) else fallback
```

```
<font face="Courier" color="#dc2626"><b>`</b></font>
```

Ganho esperado: Sanitização efetiva de mensagens de erro em produção.

Solução recomendada: Substituir is_safe_message(fallback) por is_safe_message(message).

ALTO-6: Bare Excepts com except: (sem tipo)

Arquivos:

- **apps/core/services/voice/detector.py** (linha 673-675)

- **apps/core/services/voice/whatsapp_reply.py** (linha 258)

- **apps/core/services/voice/quick_transcriber.py** (linha 90)

- **apps/core/startup.py** (linhas 101-102, 108-109)

Risco: ALTO

O problema: Blocos **except: sem especificar a exceção capturam **BaseException**, que inclui **KeyboardInterrupt**, **SystemExit**, **GeneratorExit**. Isso pode tornar o processo impossível de matar.**

Impacto prático: Um **Ctrl+C pode ser silenciado e o processo continua rodando. Debugging difícil.**

O que vai mudar: Substituir **except: por **except Exception:** em todos os locais.**

Ganho esperado: Processo ainda responsivo a sinais do sistema.

MÉDIO-5: **ChatInput.tsx — 636 Linhas com Múltiplas Responsabilidades**

Arquivo: **apps/momai/src/renderer/src/components/chat/ChatInput.tsx (636 linhas)**

Risco: MÉDIO

O problema: O componente acumula responsabilidades demais: input de texto, autocomplete, voice settings, call mode, gravação de voz, histórico de navegação, atalhos globais de teclado.

Solução recomendada: Extrair `VoiceSettingsDropdown`, `ChatToolbar`, `ChatTextInput` como componentes separados.

MÉDIO-6: `useAppInitialization.ts` — 244 Linhas, 9 `useEffects`

Arquivo: `apps/momai/src/renderer/src/hooks/useAppInitialization.ts` (244 linhas)

Risco: **MÉDIO**

O problema: Um hook que mistura onboarding, settings sync, extensões, daily briefing.

Solução recomendada: Extrair para `useWelcomeState`, `useSettingsSync`, `useDailyBriefing`.

MÉDIO-7: `@ts-ignore` em 10 Locais

Arquivos: Vários (`useChat.ts`, `useSettingsCard.ts`, `useStatus.ts`, `useWindowMaximized.ts`)

Risco: **MÉDIO**

O problema: `@ts-ignore` silencia erros de tipo sem justificativa documentada.

Solução recomendada: Corrigir tipos subjacentes ou substituir por `@ts-expect-error` com comentário.

MÉDIO-8: Timer/Interval Sem Limpeza Apropriada

Arquivos: `App.tsx` (linha 94), `useChatHandlers.ts`

(linha 101), e outros

Risco: **MÉDIO**

O problema: `setTimeout` em `App.tsx:94` não faz cleanup no unmount. Timeouts dentro de callbacks de reducer podem disparar após componente desmontado.

Solução recomendada: Retornar cleanup em todos os `useEffect` que criam timers. Criar hook `useTimeout`/`useInterval`.

MÉDIO-9: Download de Modelo TTS Sem Timeout

Arquivo: `apps/core/services/voice/tts.py` (linhas 257-259)

Risco: **MÉDIO**

O problema: `urllib.request.urlretrieve(url, model_path)` não tem timeout. Se o download travar, o TTS nunca fica pronto.

Solução recomendada: Usar `httpx.AsyncClient` com timeout + streaming. Validar checksum do download.

MÉDIO-10: Rota Síncrona Bloqueante por 30s

Arquivo: `apps/core/api/routes/voice.py` (linhas 128-145)

Risco: **MÉDIO**

O problema: `quick_transcribe()` é uma rota síncrona (`def`) que pode bloquear por até 30 segundos, consumindo um thread do pool.

Solução recomendada: Usar `async def` com `asyncio.to_thread()` ou `run_in_executor`.

MÉDIO-11: Monkey-patching de `threading.Thread.start`

Arquivo: `apps/core/runtime.py` (linhas 111-127)

Risco: **MÉDIO**

O problema: O código faz monkey-patch global de `threading.Thread.start`, afetando TODAS as threads do processo incluindo de bibliotecas terceiras. Compara mensagem de erro por string, que é frágil entre versões do Python.

Solução recomendada: Corrigir as race conditions na fonte em vez de suprimir sintomas globalmente.

MÉDIO-12: Sem Modelo Pydantic para `/chat/speak`

Arquivo: `apps/core/api/routes/chat_voice.py` (linha 24)

Risco: **MÉDIO**

O problema: `data: dict` como tipo sem validação Pydantic. Payload malformatado retorna 500 em vez de 422.

Solução recomendada: Definir `SpeakRequest(BaseModel)` com `text: str = Field(..., max_length=10000)`.

MÉDIO-13: Duplicação de Código (Audio Callbacks, Whisper Loading)

Arquivos: `detector.py`, `quick_transcriber.py`, `whatsapp_reply.py`, `voice.py`

Risco: **MÉDIO**

O problema: O padrão de callback de áudio do `sounddevice` + gerenciamento de fila é copiado em 3 arquivos. Carregamento do Whisper é duplicado em 2 funções.

Solução recomendada: Extrair `AudioStreamManager` e `WhisperModelManager` como singletons.

MÉDIO-14: Scripts de Build com Senha Hardcoded

Arquivos: `apps/momai/scripts/hydrate-bin.ps1`, `apps/momai/scripts/ensure-cert.js`

Risco: **MÉDIO**

O problema: O script `hydrate-bin.ps1` baixa binários de URLs não-pinadas. Sem verificação de checksum, um ataque ao repositório de origem comprometeria o build.

Solução recomendada: Verificar SHA256 dos binários baixados contra valor conhecido.

5. Problemas de Performance

MÉDIO-1: Verificação de GPU Repetida

Arquivos: `detector.py:168`, `voice.py:115`, `voice.py:296`

Risco: **MÉDIO**

O problema: `ctranslate2.get_cuda_device_count()` é chamado em 3 locais independentes, cada um levando centenas de milissegundos.

Ganho esperado: Redução de ~1-2s no startup.

MÉDIO-2: Debounce de Save sem Ack

Arquivo: `apps/momai/scripts/no-de-core/infrastructure/store.js` (linhas 102-121)

Risco: **MÉDIO**

O problema: Salvamento com debounce de 2s. Se o processo crashar dentro da janela, dados são perdidos.

Ganho esperado: Dados de conversação não perdidos em crash.

6. Problemas de Arquitetura

MÉDIO-1: Eventos Customizados Sem Sistema Centralizado

Impacto: 40+ eventos customizados no `window` sem registro centralizado.

Solução recomendada: Criar `EventBus` tipado.

MÉDIO-2: Estado Global Mutável Compartilhado

Arquivo: `apps/momai/scripts/no-de-core/services/shared-state.js`

Impacto: Race conditions em `Object.assign` de estado compartilhado entre módulos.

7. Problemas de Configuração

BAIXO-1: `<font face="Courier" color="#dc2626"><b>TOKENIZERS_PARALLELISM</b></font>` setado em 2 locais

`<b>Arquivo:</b> <font face="Courier" color="#dc2626"><b>tts.py:20</b></font>, <font face="Courier" color="#dc2626"><b>main.py:54</b></font>`

`<b>Impacto:</b>` Redundante.

BAIXO-2: `<font face="Courier" color="#dc2626"><b>pydantic</b></font>` listado como dependência não utilizada no forscript

`<b>Arquivo:</b> <font face="Courier" color="#dc2626"><b>apps/fortscript/pyproject.toml</b></font>`

`<b>Impacto:</b>` 1 dependência desnecessária.

BAIXO-3: Banco de dados em local imprevisível

`<b>Arquivo:</b> <font face="Courier" color="#dc2626"><b>apps/core/database/models.py</b></font> (linhas 53-59)`

`<b>Impacto:</b>` Se `<font face="Courier" color="#dc2626"><b>MOMAI_DATA_DIR</b></font>` não for setado, DB fica na árvore de source.

8. Tabela Resumo de Prioridades

| ID | Problema | Categoria | Gravidade | Arquivo |

|----|-----|-----|-----|-----|

| CR-1 | Credenciais Trello commitadas | Segurança | `<b>CRÍTICO</b>` | `<font face="Courier" color="#dc2626"><b>apps/momai/.env</b></font>` |

| CR-2 | Senha PFX hardcoded | Segurança | `<b>CRÍTICO</b>` | `<font face="Courier" color="#dc2626"><b>ensure-cert.js</b></font>`, `<font face="Courier" color="#dc2626"><b>sign-appx.js</b></font>` |

| A-1 | Chaves API expostas no renderer | Segurança | **ALTO** |
useSettingsCard.ts |

| A-2 | Proxy Python sem auth | Segurança | **ALTO** | chat.routes.js, index.js |

| A-3 | IPC genérico no preload | Segurança | **ALTO** | preload/index.ts |

| A-4 | Token na linha de comando | Segurança | **ALTO** | >windowManager.ts |

| A-5 | Token vazado para extensões | Segurança | **ALTO** | extension-host-manager.js |

| A-6 | Command injection em ZIP | Segurança | **ALTO** | extensions.routes.js |

| A-7 | Shutdown sem autenticação | Segurança | **ALTO** | status.routes.js |

| A-8 | Checksum bypass | Segurança | **ALTO** | extensions.routes.js |

| A-9 | Uso massivo de any | Qualidade | **ALTO** | 25+ arquivos TS |

| A-10 | Zero error boundaries | Qualidade | **ALTO** | Todo o renderer |

| A-11 | Prop drilling massivo | Qualidade | **ALTO** | App → ... → Container |

| A-12 | Handler monolítico 619 linhas | Qualidade | **ALTO** | useChatHandlers.ts |

| A-13 | sanitize_message bug | Qualidade | **ALTO** | error_handler.py |

| A-14 | Bare excepts | Qualidade | **ALTO** | detector.py, whatsapp_reply.py |

| M-1 | Token WS na query string | Segurança | **MÉDIO** | vários |

| M-2 | CORS file:// | Segurança | MÉDIO | core/main.py |

| M-3 | Sem CSP | Segurança | MÉDIO | >windowManager.ts |

| M-4 | Timing attack no token | Segurança | MÉDIO | auth.py |

| M-5 | ChatInput 636 linhas | Qualidade | MÉDIO | ChatInput.tsx |

| M-6 | useAppInit 244 linhas | Qualidade | MÉDIO | useAppInitialization.ts |

| M-7 | @ts-ignore em 10 locais | Qualidade | MÉDIO | vários |

| M-8 | Timers sem cleanup | Qualidade | MÉDIO | vários |

| M-9 | Download TTS sem timeout | Performance | MÉDIO | tts.py |

| M-10 | Rota síncrona 30s | Performance | MÉDIO | voice.py |

| M-11 | Monkey-patch Thread | Qualidade | MÉDIO | runtime.py |

| M-12 | Sem Pydantic no /chat/speak | Qualidade | MÉDIO | chat_voice.py |

| M-13 | Código duplicado | Qualidade | MÉDIO | 4 arquivos |

| M-14 | GPU check repetido | Performance | MÉDIO | 3 locais |

| M-15 | Debounce save sem ack | Performance | MÉDIO | store.js |

| M-16 | Eventos sem centralização | Arquitetura | MÉDIO | 40+ eventos |

| M-17 | Estado mutável compartilhado | Arquitetura | MÉDIO | shared-state.js |

| B-1 | TOKENIZERS_PARALLELISM duplicado | Config | BAIXO | 2 locais |

| B-2 | pydantic não usado | Config | BAIXO | fortscript/pyproject.toml |

| B-3 | DB em local imprevisível | Config | **BAIXO** | `database/models.py` |

| B-4 | TitleBar props não usadas | Qualidade | **BAIXO** | `TitleBar.tsx` |

| B-5 | Icon maps duplicados | Qualidade | **BAIXO** | `LateralBar`, `ExtensionsView` |

| B-6 | Re-export depreciado | Qualidade | **BAIXO** | `MessageItem.tsx` |

| B-7 | Números mágicos | Qualidade | **BAIXO** | `detector.py` |

| B-8 | TypedDict vs dataclass | Qualidade | **BAIXO** | `fortscript/main.py` |

| B-9 | Loop busy-wait sem graceful exit | Qualidade | **BAIXO** | `fortscript/main.py` |

9. Recomendação Final

Prioridade Absoluta (Corrigir Imediatamente)

1. **Revogar credenciais comprometidas** — Token Trello e certificado PFX. Remover do Git history.

2. **Restringir IPC do preload** — Remover `invoke`, `send`, `on` genéricos. Criar funções nomeadas seguras.

3. **Adicionar auth header ao proxy Node Core → Python** — Sem isso, o pipeline de voz está quebrado.

4. **Remover token de sessão de command-line args** — Mover para IPC pós-carregamento.

5. **Parar de vazar token para extensões** — Extensões não precisam do token mestre.

Curto Prazo (Próximo Sprint)

6. **Adicionar Error Boundaries** no React — 3 níveis: App, views, message list.

7. **Iniciar migração de tipos** — Ativar `strict` no tsconfig, definir interfaces para os tipos mais críticos.

8. **Refatorar** `useChatHandlers.ts` — Extrair handlers por tipo de mensagem.

9. **Corrigir** `sanitize_message` — Verificar mensagem real em vez do fallback.

10. **Substituir** `exec()` por `adm-zip` — Eliminar command injection nas extensões.

Médio Prazo (Próximos Sprints)

11. **Introduzir React Context** para eliminar prop drilling do chat.

12. **Extrair** `ChatInput` em componentes menores.

13. **Centralizar eventos** em um EventBus tipado.

14. **Adicionar timeouts** a todos os downloads de modelo.

15. **Substituir bare excepts** por `except Exception:`.

Benefícios Esperados

- **Segurança:** Eliminação de 11 vetores de ataque, incluindo 2 críticos

- **Manutenibilidade:** Redução de complexidade em ~40% nos módulos mais problemáticos

- **Performance:** Ganho de ~2-3s no startup (GPU check + downloads)

- **Resiliência:** Queda de 100% para ~10% em cenários de erro (Error Boundaries)

- **Qualidade:** Redução de ~90% no uso de `any`, bugs de tipo capturados em compile-time
